Advanced Stack Storage System

Project 8 — Digital Logic Design

LogicWorks Circuit Implementation

1. Project Title and Objective

Title: Advanced Stack Storage System

Objective: To design and implement a modified stack data structure in LogicWorks based on the Last In First Out (LIFO) principle. Unlike a conventional stack, this system automatically discards the oldest element when the stack is full and a new value is pushed, ensuring the stack always holds the most recent values up to its maximum capacity of 4 elements.

2. Introduction

A stack is a fundamental data structure that follows the Last In First Out (LIFO) principle — the last element inserted is the first to be removed. This project extends the standard stack by introducing a circular buffer mechanism that allows the stack to accept new data even when full, automatically removing the oldest entry.

For example, if the stack has a capacity of 4 and contains the values 2, 4, 6, and 8, pushing the value 10 causes 2 (the oldest) to be discarded. The stack then contains 4, 6, 8, and 10. Popping all four values returns them in LIFO order: 10, 8, 6, 4.

The system is implemented entirely in LogicWorks using combinational and sequential logic components including D flip-flops, multiplexers, decoders, counters, and logic gates.

3. Working and Functionality of Each Operation

3.1 Push Operation

When the user activates the Push switch and toggles the clock:

- The HEAD counter increments, pointing to the next register slot.
- The 2x4 decoder activates the corresponding register via its enable logic.

- The quad 2x1 multiplexer passes the data input switches to the D inputs of the selected register.
- On the rising clock edge, the selected register stores the 4-bit value.
- The size counter increments if the stack is not full.
- If the stack is full (size = 4), the size counter is blocked and HEAD simply overwrites the oldest slot, effectively discarding it.

3.2 Pop Operation

When the user activates the Pop switch and toggles the clock:

- The UP pin of both HEAD and size counters is driven by the Push switch. When Push=0 and Pop=1, UP=0, causing both counters to count down.
- A 4-bit adder computes HEAD-1 by adding 1111 (two's complement of -1) to the HEAD counter outputs. Only the lower two bits (S0, S1) of the adder output are used to select from 4 registers.
- This adder output feeds the 4-to-1 output multiplexers, which select and display the correct register value as the Popped Value.
- A special wraparound case exists: when HEAD=0 and Pop=1, decrementing would underflow to 15. To handle this, a 4-input NOR gate detects HEAD=0, ANDs it with Pop, and uses the result (through a NOT gate) to trigger the LD pin of the HEAD counter, loading the value 3 (0011) directly.
- The HEAD counter decrements on each pop. The size counter also decrements, but only through its independent clock path.
- If the stack is empty (size = 0), the Empty indicator activates.

3.3 Overflow Handling

When the stack is full and a push is performed:

- The Full indicator (size = 4) is active.
- The NOT Full logic blocks the size counter clock, keeping size at 4.
- HEAD still increments and wraps around (mod-4), overwriting the oldest register slot.
- The new value replaces the oldest, maintaining a rolling window of the 4 most recent values.

3.4 Empty Detection

A 3-input NOR gate monitors Q0, Q1, and Q2 of the size counter. When all are 0 (size = 0), the Empty indicator outputs logic 1, signaling that the stack has no elements.

3.5 Full Detection

The Full indicator is driven by the Boolean expression Q2 AND NOT(Q1) AND NOT(Q0) on the size counter outputs, which evaluates to 1 only when size = 4 (binary 100).

4. Circuit Design / Block Diagram

The circuit is organized into the following functional blocks:

Block	Components Used	Function
Register File	16 D flip-flops (4 registers × 4 bits)	Stores the 4 stack elements
Write Enable Logic	2x4 Decoder, NOT gates, AND gates, Quad 2x1 MUX	Selects which register to write on push
HEAD Counter	4-bit Up/Down Counter, OR gate, AND gate	Tracks write position; increments on push, decrements on pop
HEAD Mod-4 Wraparound	NOT gates, 3-input AND gate, OR gate → CLR	Resets HEAD to 0 when it reaches 4 (Q2 AND NOT Q1 AND NOT Q0)
HEAD Pop Wraparound	4-input NOR gate, AND gate, NOT gate → LD pin	Loads 3 into HEAD when HEAD=0 and Pop=1 to prevent underflow
Size Counter	4-bit Up/Down Counter, independent AND/OR/NOT gates	Tracks number of elements; blocked from incrementing past 4
Full Detector	2 NOT gates, 3-input AND gate	Asserts Full when size = 4 (Q2 AND NOT Q1 AND NOT Q0)
Empty Detector	3-input NOR gate	Asserts Empty when size = 0
Output MUX	4× 4-to-1 Multiplexers	Reads correct register on pop based on adder output
HEAD-1 Adder	4-bit Adder (adds 1111 to HEAD)	Computes HEAD-1 in two's complement; S0,S1 feed mux select
Separate Clock Logic	Two independent AND gates	HEAD counter clocks on Push OR Pop; size counter clocks on (Push AND NOT Full) OR Pop

5. Explanation of Inputs and Outputs

5.1 Inputs

Input	Type	Description
Data Input (D0–D3)	4 switches	4-bit value to push (0–15)
Push	Switch	Triggers a push operation on clock edge
Pop	Switch	Triggers a pop operation on clock edge
Clock (CLK)	Switch	Manual clock — rising edge triggers operation
Reset (R)	Switch	Active-low reset for all flip-flops
Clear (CLR)	Switch	Resets HEAD and size counters to 0

5.2 Outputs

Output	Type	Description
Stack Contents (R0–R3)	16 probes	Shows all 4 stored values (4 bits each)
Popped Value	4 probes	Displays the value removed on pop
Full Indicator	1 probe	Logic 1 when stack has 4 elements
Empty Indicator	1 probe	Logic 1 when stack has 0 elements

6. Simulation Results

The following test was performed to verify correct operation:

Operation	Value Pushed	Stack Contents (Top→Bottom)	HEAD	Size	Full	Empty
Reset	—	Empty	0	0	0	1
Push	0010 (2)	2	1	1	0	0
Push	0100 (4)	4, 2	2	2	0	0
Push	0110 (6)	6, 4, 2	3	3	0	0
Push	1000 (8)	8, 6, 4, 2	0	4	1	0
Push (overflow)	1010 (10)	10, 8, 6, 4	1	4	1	0
Pop	—	8, 6, 4	0	3	0	0
Pop	—	6, 4	3	2	0	0
Pop	—	4	2	1	0	0

Pop	—	Empty	1	0	0	1

Results confirmed correct LIFO behavior, automatic overflow handling, and accurate Full/Empty detection.

7. Challenges Faced and Conclusion

7.1 Challenges

- Active-low pins (S, R on flip-flops; EN on decoder and MUX; LD on counters) required careful handling. Connecting R to VCC instead of a reset switch meant flip-flops could not be initialized, causing X (unknown) states throughout the circuit.
- The HEAD counter needed mod-4 wraparound. A combinational reset using Q2 AND NOT(Q1) AND NOT(Q0) fed through an OR gate into CLR successfully resets HEAD to 0 when it reaches 4.
- Pop underflow was a major challenge. When HEAD=0, decrementing would go to 15 instead of 3. This was resolved using the LD pin: a 4-input NOR gate detects HEAD=0, ANDs it with Pop, and triggers LD to load 3 (0011) directly into the counter.
- Pop read timing was off-by-one since HEAD points to the next empty slot after each push. A 4-bit adder computing HEAD-1 (by adding 1111 in two's complement) and feeding S0,S1 to the output muxes resolved this correctly.
- The HEAD counter and size counter initially shared the same clock AND gate. This caused HEAD to stop incrementing during overflow because the NOT Full logic was blocking both. Splitting them into two independent clock paths fixed this.
- The LD pin on the HEAD counter conflicted with the mod-4 reset since both needed to control the same pin. Using CLR for the overflow reset (HEAD=4→0) and LD exclusively for the pop wraparound (HEAD=0→3) resolved the conflict cleanly.
- Uninitialized flip-flop states (X) required a dedicated reset switch on all R pins. Initially R was connected to VCC which prevented any manual reset.

7.2 Conclusion

The Advanced Stack Storage System was successfully designed and simulated in LogicWorks. The circuit correctly implements LIFO push and pop operations, handles overflow by discarding the oldest element, and accurately detects full and empty conditions. The design demonstrates how classical data structures can be realized using fundamental digital logic components, and reinforces key concepts in sequential

circuit design including counter control, multiplexed data paths, and combinational enable logic.